

Linked Lists

Notes on the ADT and its four common shapes, with Java and C/C++ examples

A word before you read. These are my notes from years of teaching this unit, cleaned up so they are readable. While C/C++ was my first language, I am a bit rusty because I have taught Java for years now. For these notes, I have been using AI to help type these up, check for issues, and fill in explanations, which means a wrong or imprecise statement can slip in and survive longer than it should, and I do not fully trust the AI, so I am still editing. They are **not** meant to be a study guide, they are not gospel, and not everything here shows up in a lecture (and some lecture items are not here). If you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, **tell me**. You will be doing students and me a favor, and you will understand it better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. We are a two-language course, thus I will do my best to show everything in Java (**CIS**) and C/C++ (**CS**).

1. The ADT wall

An abstract data type, or ADT, describes a collection of data together with the operations allowed on it, without saying how those operations are actually carried out. Think of it as a wall between the program and the data structure. The program asks for an operation, such as add or remove, and the data structure carries it out on the other side of the wall. The program never needs to know how the wall does its job, only that it will. This separation is called information hiding, and it is what lets you swap one implementation of an ADT for another without changing any code that uses it.

A linked list is one implementation of the list ADT: a collection of items where each item knows only about its neighbor. Common ADTs you will meet later in the course, including stacks, queues, tables, trees, and graphs, are all built out of the same basic idea of items connected to one another, so the vocabulary and habits you build here carry forward.

2. Terminology

- **Node.** A single unit in the list. It holds a piece of data and a reference, or link, to the next node.
- **Head.** The first node in the list. It has no predecessor.
- **Tail.** The last node in the list. In a singly or doubly linked list it has no successor. Some implementations also keep a separate tail reference so the last node can be reached without walking the whole list.
- **Predecessor and successor.** For a given node, the predecessor is the node before it and the successor is the node after it.
- **Sequential access.** You reach a node by following links from the head, one at a time. There is no way to jump directly to the middle of the list the way you can with an array index.

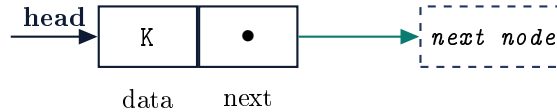


Figure 1: Anatomy of one node. A data field holds the value; a next field points to the following node. The head reference marks the first node in the list, not a field inside any node.

3. Array-based versus pointer-based

A list can be built two ways. An array-based list stores items in a fixed block of contiguous memory and accesses them by index. A pointer-based, or reference-based, list is what most people mean by a linked list: items are scattered in memory, and each one carries a reference to the next.

Table 1: Array-based versus pointer-based implementations of a list.

Aspect	Array-based	Pointer-based
Memory layout	Contiguous, one block	Scattered, one node at a time
Access by position	Direct, by index	Sequential, walk from the head
Insert or remove in the middle	Shift every element after the gap	Relink two pointers, nothing moves
Size	Often fixed, or must be resized	Grows and shrinks one node at a time
Strength	Fast random access	Fast insert and remove once you are there

Java hides the address of an object reference from you, so a linked list built in Java still uses pointers under the hood, you simply cannot print or compute with the raw address. C and C++ let you see that address directly, which is part of why the same idea looks more mechanical in C/C++ code than in Java code.

4. Four shapes of a linked list

The same node idea can be arranged four ways.

- **Singly linked.** Each node points to its successor only. You can only walk forward, and the last node's next field is null.
- **Circular linked.** Like a singly linked list, except the last node points back to the first node instead of to null. There is no true end, so code that walks the list has to stop on a condition other than "next is null," usually by checking whether it has arrived back at the starting node.
- **Doubly linked.** Each node carries two links, one to its successor and one to its predecessor, so you can walk in either direction. The extra link costs memory per node but makes some operations, such as removing a node you already have a reference to, much cheaper.
- **Circular doubly linked.** Combines both ideas: two links per node, and the ends wrap around to each other in both directions. Some implementations add a dummy head node, a node that holds no real data and simply anchors the ring, so that insertion and removal logic does not need a special case for an empty list.

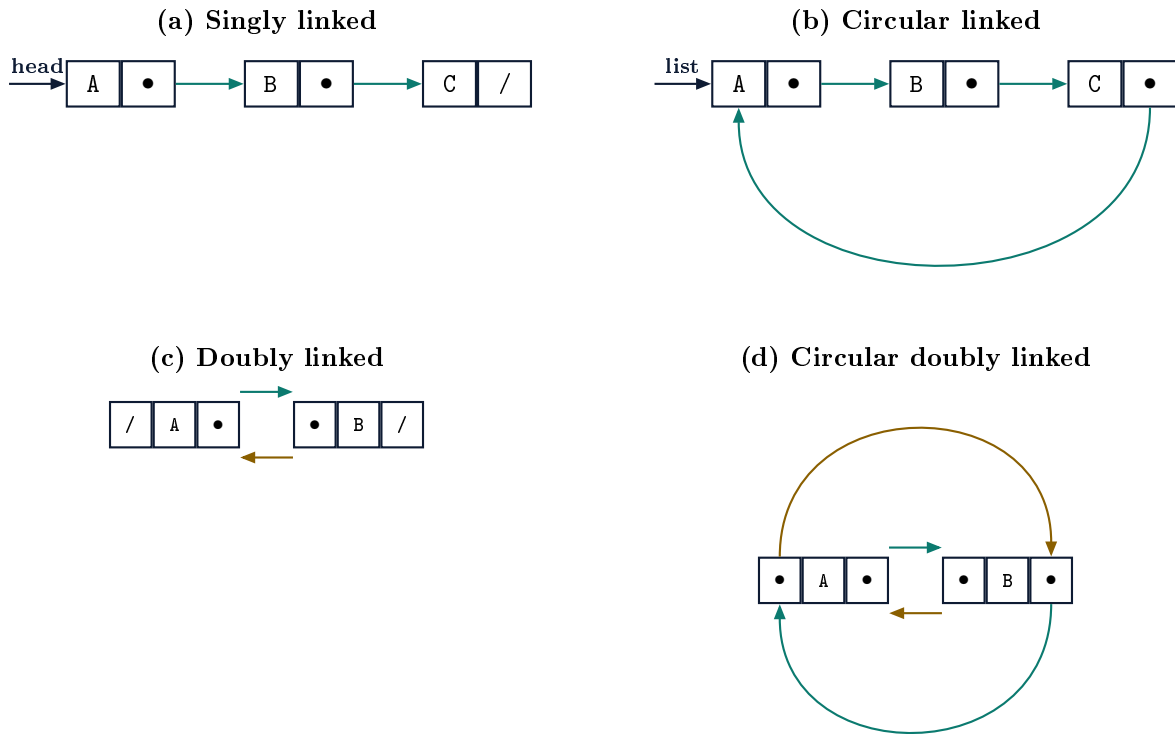


Figure 2: The four shapes a linked list can take. Teal arrows are next pointers; amber arrows are prev pointers; a slash marks a null reference.

Table 2: Quick comparison of the four shapes.

Type	Traversal	Links per node	Typical use
Singly linked	Forward only	1	The default choice when you only ever walk forward
Circular linked	Forward, wraps around	1	Round robin scheduling, anything that cycles
Doubly linked	Forward and backward	2	You need to back up, such as an undo history
Circular doubly linked	Forward and backward, wraps around	2	Most flexible, such as a playlist that loops both ways

A note on the code. The Java and C/C++ listings in these notes are not the exact code copied from either textbook. I often attempt to simplify the code for teaching purposes. Names, structure, even which operations a class exposes can differ from what the book shows you, or what I show you in class. Learning to look at someone else's version of a familiar idea and recognize that it does the same job as your version is its own skill, one you will need constantly once you leave this course. When the versions genuinely disagree, the textbook and the compiler settle it, not these notes.

5. Code: building a singly linked list

The examples below build a working singly linked list: add to the front, retrieve by position, remove by value, display, walk the chain of links directly, and destroy. Add to the front is the easiest place to start because it only touches the head reference and the new node, nothing else in the list has to change; the rest build outward from there.

Java

Node.java

```
public class Node<T> {
    private T data;
    private Node<T> next;

    public Node(T data) {
        this.data = data;
        this.next = null;
    }

    public T getData() { return data; }
    public void setData(T data) { this.data = data; }
    public Node<T> getNext() { return next; }
    public void setNext(Node<T> next) { this.next = next; }
}
```

LinkedList.java

```
public class LinkedList<T> {
    private Node<T> head;

    public boolean isEmpty() {
        return head == null;
    }

    // Adds to the front. Touches only head, so this is O(1).
    public void addFirst(T value) {
        Node<T> newNode = new Node<>(value);
        newNode.setNext(head);
        head = newNode;
    }

    // 0-based position.
    public T retrieve(int position) {
        if (position < 0) {
            throw new IndexOutOfBoundsException("position must be non-negative");
        }
        Node<T> current = head;
        int index = 0;
        while (current != null) {
            if (index == position) {
                return current.getData();
            }
        }
    }
}
```

```

        current = current.getNext();
        index++;
    }
    throw new IndexOutOfBoundsException("position is past the end of the list");
}

// Removes the first node whose data equals value. Relies on T's equals().
public boolean remove(T value) {
    if (isEmpty()) {
        return false;
    }
    if (head.getData().equals(value)) {
        head = head.getNext();
        return true;
    }
    Node<T> current = head;
    while (current.getNext() != null && !current.getNext().getData().equals(value))
    {
        current = current.getNext();
    }
    if (current.getNext() == null) {
        return false;    // value was not found
    }
    current.setNext(current.getNext().getNext());
    return true;
}

public void display() {
    Node<T> current = head;
    while (current != null) {
        System.out.print(current.getData() + " ");
        current = current.getNext();
    }
    System.out.println();
}

// Walks the list and prints each node's own identity, its data, and the
// identity of the node it points to next.
public void walk() {
    Node<T> current = head;
    System.out.println("Head:" + head);
    while (current != null) {
        System.out.printf("%-24s %-11s %s\n", current, current.getData(), current.
getNext());
        current = current.getNext();
    }
    System.out.println();
}

// Empties the list. Nothing references the old nodes afterward, so the
// garbage collector reclaims them; there is no delete to call.
public void destroy() {
    head = null;
}

```

```
}
```

Java manages the memory for you. Once no reference anywhere in the program points at a node, the garbage collector reclaims it. You never call anything like `delete`. Notice also that `walk` prints a null next reference as the word **null** with no extra work, Java gives you that for free.

C/C++

LinkedList.h

```
template <typename T>
struct Node {
    T data;
    Node<T>* next;

    Node(const T& value) : data(value), next(nullptr) {}
};

template <typename T>
class LinkedList {
private:
    Node<T>* head;

    // Small helper used only by walk(). Unlike Java, printing a null
    // pointer in C++ gives you 0, not a readable label, unless you check
    // for it yourself.
    static std::string ptrLabel(const Node<T>* p) {
        if (p == nullptr) {
            return "nullptr";
        }
        std::ostringstream oss;
        oss << p;
        return oss.str();
    }

public:
    LinkedList() : head(nullptr) {}

    bool isEmpty() const {
        return head == nullptr;
    }

    // Adds to the front. Touches only head, so this is O(1).
    void addFirst(const T& value) {
        Node<T>* newNode = new Node<T>(value);
        newNode->next = head;
        head = newNode;
    }

    // 0-based position.
    T retrieve(int position) const {
        if (position < 0) {
            throw std::out_of_range("position must be non-negative");
        }
    }
};
```

```

    }
    Node<T>* current = head;
    int index = 0;
    while (current != nullptr) {
        if (index == position) {
            return current->data;
        }
        current = current->next;
        index++;
    }
    throw std::out_of_range("position is past the end of the list");
}

// Removes the first node whose data equals value. Relies on T's operator==.
bool remove(const T& value) {
    if (isEmpty()) {
        return false;
    }
    if (head->data == value) {
        Node<T>* toDelete = head;
        head = head->next;
        delete toDelete;
        return true;
    }
    Node<T>* current = head;
    while (current->next != nullptr && !(current->next->data == value)) {
        current = current->next;
    }
    if (current->next == nullptr) {
        return false;    // value was not found
    }
    Node<T>* toDelete = current->next;
    current->next = toDelete->next;
    delete toDelete;
    return true;
}

void display() const {
    Node<T>* current = head;
    while (current != nullptr) {
        std::cout << current->data << " ";
        current = current->next;
    }
    std::cout << std::endl;
}

// Walks the list and prints each node's own address, its data, and the
// address it points to next.
void walk() const {
    std::cout << std::left;
    std::cout << "head: " << ptrLabel(head) << std::endl;
    Node<T>* current = head;
    while (current != nullptr) {
        std::ostringstream dataStream;

```

```

        dataStream << current->data;
        std::cout << std::setw(16) << ptrLabel(current)
                    << std::setw(14) << dataStream.str()
                    << ptrLabel(current->next) << std::endl;
        current = current->next;
    }
    std::cout << std::endl;
}

~LinkedList() {
    destroy();
}

// Frees every node and empties the list. The destructor calls this too,
// so there is exactly one place that owns the cleanup logic.
void destroy() {
    while (head != nullptr) {
        Node<T>* temp = head;
        head = head->next;
        delete temp;
    }
}
};

```

Here you own the memory. Every node you create with **new** has to be reclaimed with **delete** once nothing points at it, which is why **destroy** walks the list and deletes each node, and the destructor simply calls **destroy** so there is exactly one place that owns the cleanup logic. The current edition of the C/C++ text builds on this idea with smart pointers, such as **unique_ptr**, which call **delete** for you automatically when a node becomes unreachable. The raw-pointer version above is worth knowing either way, since it shows exactly what a smart pointer is doing on your behalf. Notice also that **walk** prints a null pointer as **0** unless you check for it yourself, the **ptrLabel** helper exists entirely to close that gap; Java's **walk** gets the word **null** for free, C/C++ does not.

These listings are simplified for the page. For the actual, compiling implementation, including the full project structure, refer to the code files provided alongside these notes rather than retyping the excerpts above.

6. Should the List Keep a Tail Reference?

The **addFirst** operation above only touches **head**, so it runs in constant time. **addLast**, which appends to the end of the list, does not: it has to walk every node to find the current last one, so its cost grows with the size of the list.

If a program appends far more than it prepends, that gap matters. The common fix is to keep a second reference, **tail**, alongside **head**, always pointing at the current last node. With a tail reference, **addLast** becomes:

```

tail.setNext(newNode);
tail = newNode;

```

which touches only two references, no matter how long the list is.

The cost is bookkeeping. Every operation that can change which node is last, appending and removing the last node, has to remember to update `tail` too. Forgetting that update does not usually crash right away; the list still looks correct for everything that never reaches the last node, which makes the bug easy to miss until something does.

We are not going to build a tail-tracking version of this class in these notes. Comparing how expensive an operation is, and what "constant time" versus "grows with the size of the list" actually means, is its own topic, and we will spend real time on it later in the semester. For now, treat this as a preview: the same operation, adding a node, can be cheap or expensive depending on what the list already knows about itself.

7. What changes for the other three shapes

You do not need a separate class for each shape, only a small, targeted change.

Doubly linked. Add a second link field, and update it on both sides of an insertion.

```
private DNode<T> prev;    // the only new field, compared to a singly linked node

// inside addFirst, after creating newNode:
newNode.setNext(head);
if (head != null) {
    head.setPrev(newNode);
}
head = newNode;
```

Circular linked. Instead of leaving the last node's next field null, point it back at the head. If you keep a tail reference, updating it after an insertion is usually cleaner than searching for the last node each time.

```
tail->next = head;    // instead of tail->next = nullptr
```

Circular doubly linked. Combine both changes: two link fields per node, and both ends wrap around to each other instead of pointing to null or nullptr.

8. Traps people fall into

- **Losing the head.** If you reassign head before you have saved a reference to the old first node somewhere the new node needs it, the rest of the list becomes unreachable. Order matters: point the new node at the old head first, then move head.
- **Forgetting the empty-list case.** Any operation that reads head's data needs to check for null or nullptr first, or it will crash on an empty list.
- **Needing the predecessor.** Removing or inserting after a specific node usually requires a reference to the node before it, not just the node itself. Walking the list one step ahead of where you actually need to stop is a common fix.
- **Infinite loops on a circular list.** A while loop that checks for null will never end on a circular list, because there is no null to find. Stop when you arrive back at the node you started from instead.

- **Updating one link and not the other.** In a doubly linked list it is easy to fix the next pointer during an insert or remove and forget the prev pointer on the neighboring node. Forward traversal will look correct while backward traversal is silently broken, so test both directions.
- **Forgetting to delete in C/C++.** Removing a node from the list does not free its memory by itself. If you are not using smart pointers, you still need to call delete on the node you just unlinked, or the memory leaks.

9. Where this lives in the textbooks

Both books use the wall to name the ADT chapter, so it is easy to find.

- **Java, 3rd edition (Prichard and Carrano).** Chapter 4, Data Abstraction: The Walls, covers the ADT itself. Chapter 5, Linked Lists, covers preliminaries, programming with linked lists, and the variations, including tail references, circular linked lists, and dummy head nodes.
- **C/C++, 8th edition (Carrano and Henry).** Chapter 1, Data Abstraction: The Walls, covers the ADT itself. The linked implementation technique appears in Chapter 4, Link-Based Implementations, paired with Chapter 3's array-based version, both built around the Bag ADT. The list ADT itself, including a linked list implementation of it, returns later in Chapters 12 and 13, Lists and List Implementations.